ÉCOLE POLYTECHNIQUE DE SOUSSE

COMPTE RENDU DE TP3

Persistance des données avec MongoDB et Mongoose

API REST Express - Relations et programmation asynchrone

Étudiante : Ferdaous Ben Taleb

Matière : MERN Stack

Année universitaire : 2026-2027

Travail individuel

1. Objectif du TP

Le but est de connecter l'API Express à MongoDB avec Mongoose. Les utilisateurs et les articles sont enregistrés dans la base de données.

2. Réponse à la question Predict

Dans le TP2, les données étaient perdues après l'arrêt du serveur. Dans le TP3, elles sont enregistrées dans MongoDB et restent disponibles après le redémarrage.

3. Configuration de MongoDB

J'ai installé mongoose et dotenv. La connexion se trouve dans config/db.js. L'adresse de la base est placée dans le fichier .env, qui est ignoré par Git.

```
npm install mongoose dotenv
```

4. Modèles et contrôleurs

Le modèle User contient name, email, age, role, bio et phone. Le modèle Article contient title, content et author. Les contrôleurs utilisent async/await pour communiquer avec MongoDB.

Chaque article garde l'ObjectId de son auteur. La méthode populate() permet d'afficher le nom et l'email de cet auteur dans la réponse.

5. Exercice 1

J'ai ajouté bio et phone. Le téléphone doit avoir huit chiffres. La suppression d'un utilisateur est refusée s'il possède déjà un article.

6. Exercice 2

Un ObjectId incorrect, comme abc, renvoie maintenant 400 Bad Request au lieu de provoquer une erreur interne du serveur.

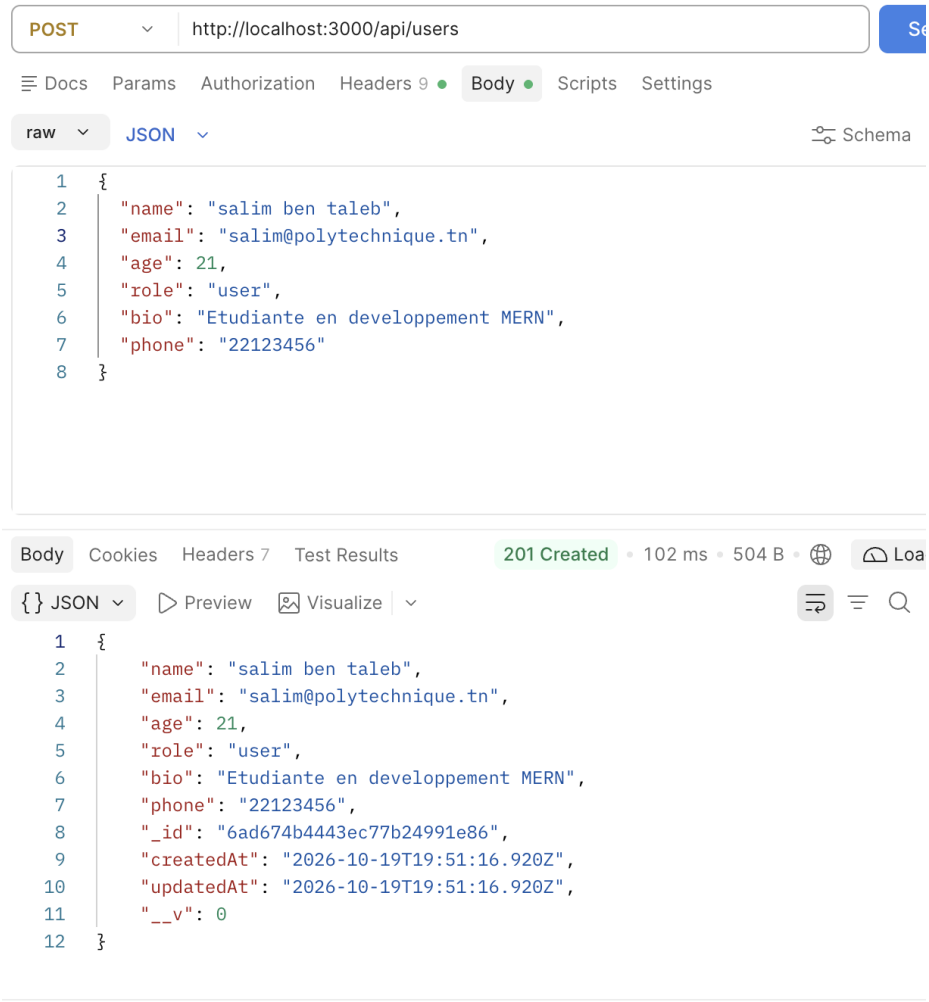
Conclusion

Les données sont maintenant conservées dans MongoDB. Mongoose permet aussi de valider les données et de mieux gérer les erreurs.

7. Tests réalisés avec Postman

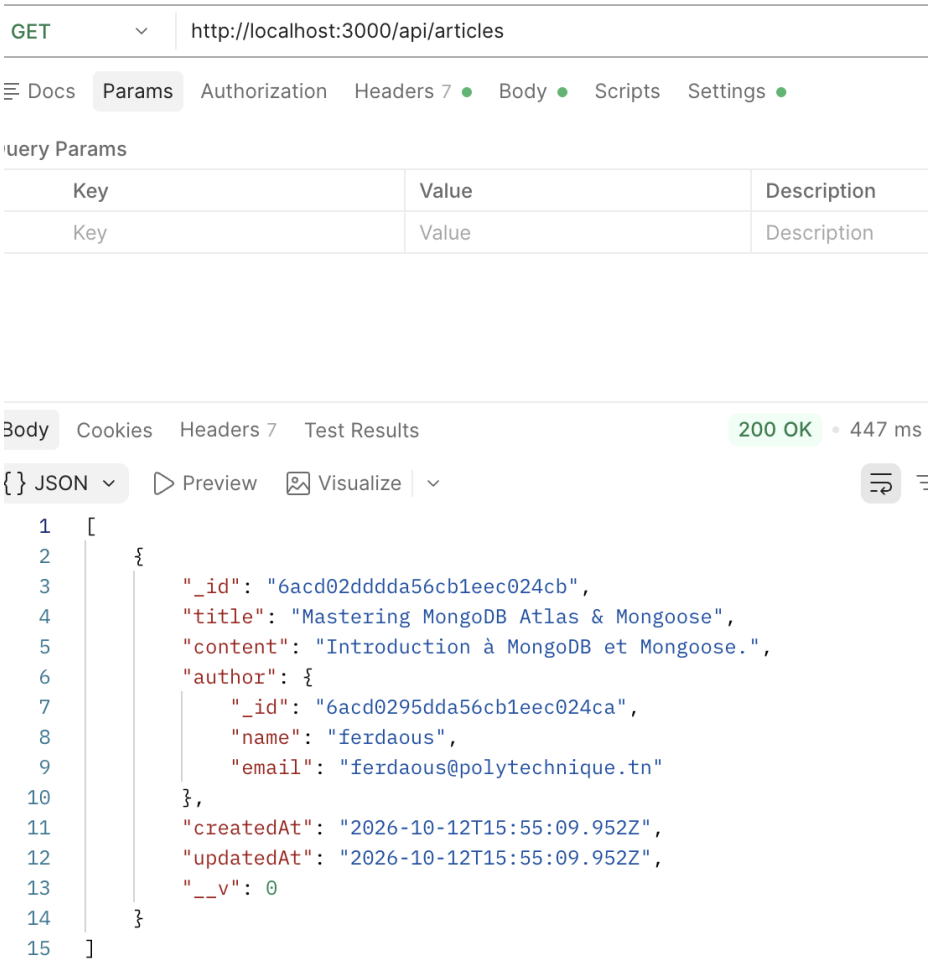
Les routes sont testées avec Postman. Chaque capture montre l'URL, la réponse et le statut HTTP.

Capture 1 - POST http://localhost:3000/api/users - 201 Created



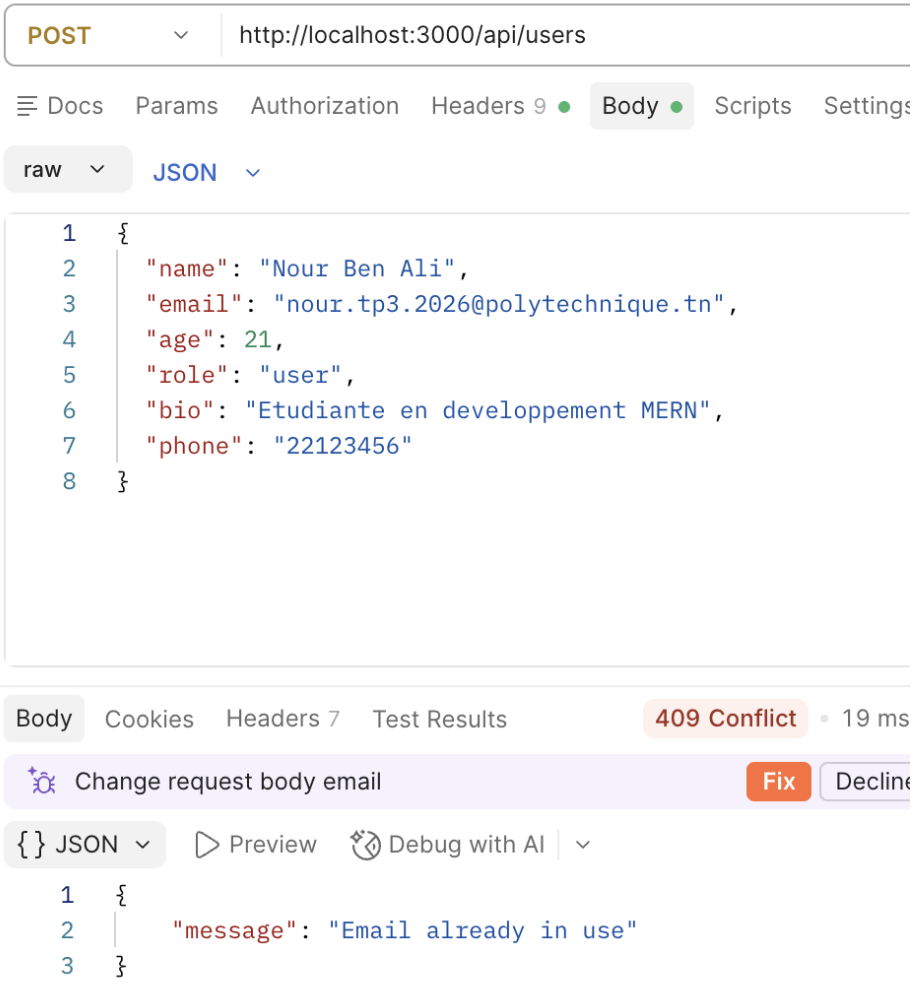
L'utilisateur est créé et enregistré dans MongoDB.

Capture 2 - GET http://localhost:3000/api/articles - 200 OK



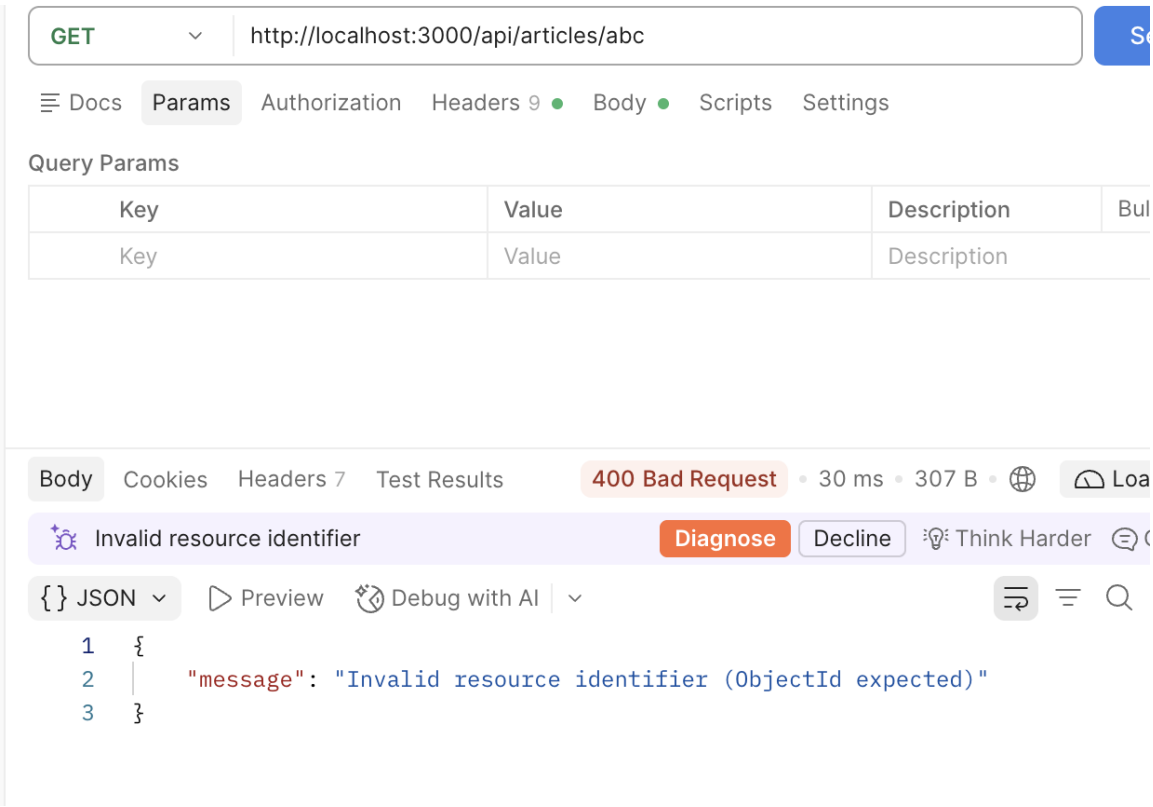
La réponse affiche l'article avec le nom et l'email de son auteur.

Capture 3 - POST /api/users avec le même email - 409 Conflict



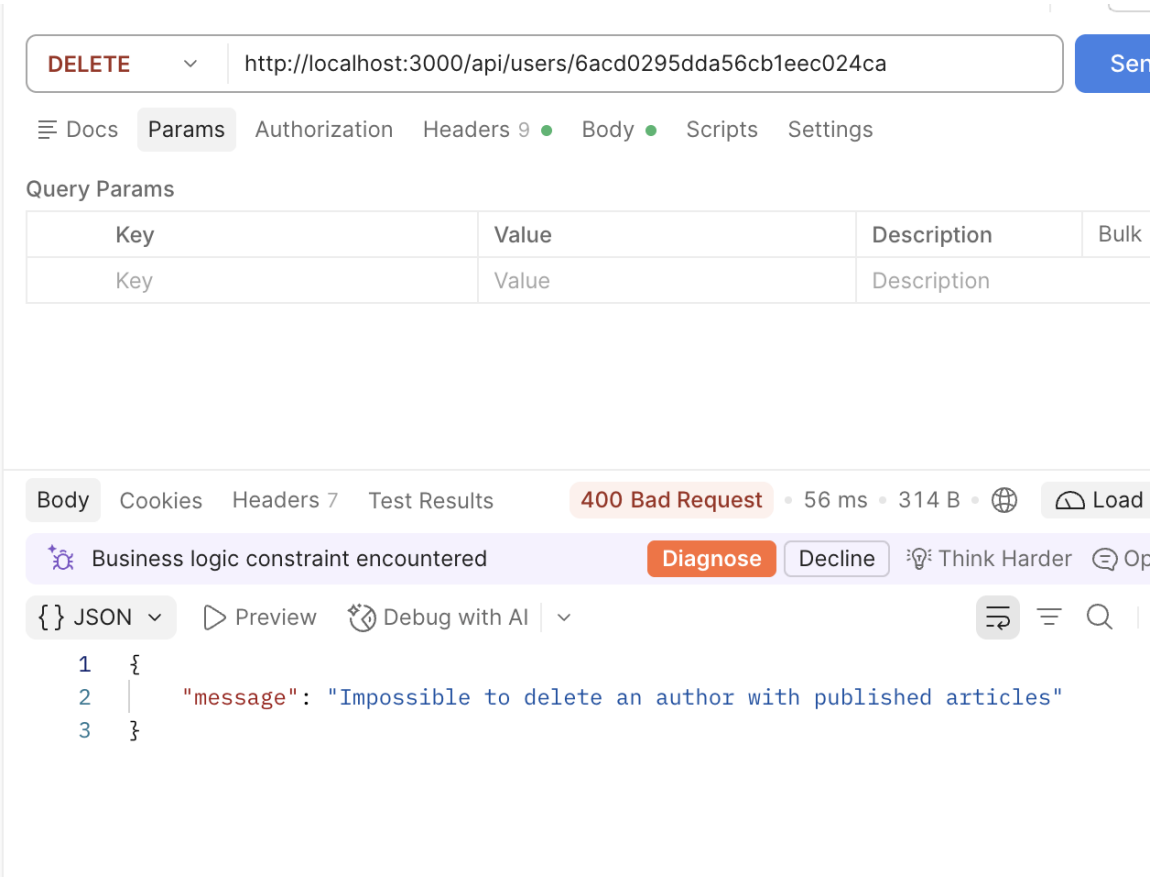
Le même email est refusé.

Capture 4 - GET /api/articles/abc - 400 Bad Request



Un identifiant incorrect renvoie 400 Bad Request.

Capture 5 - DELETE /api/users/:id - 400 Bad Request



Un auteur avec un article ne peut pas être supprimé.

8. Vérification de la base de données

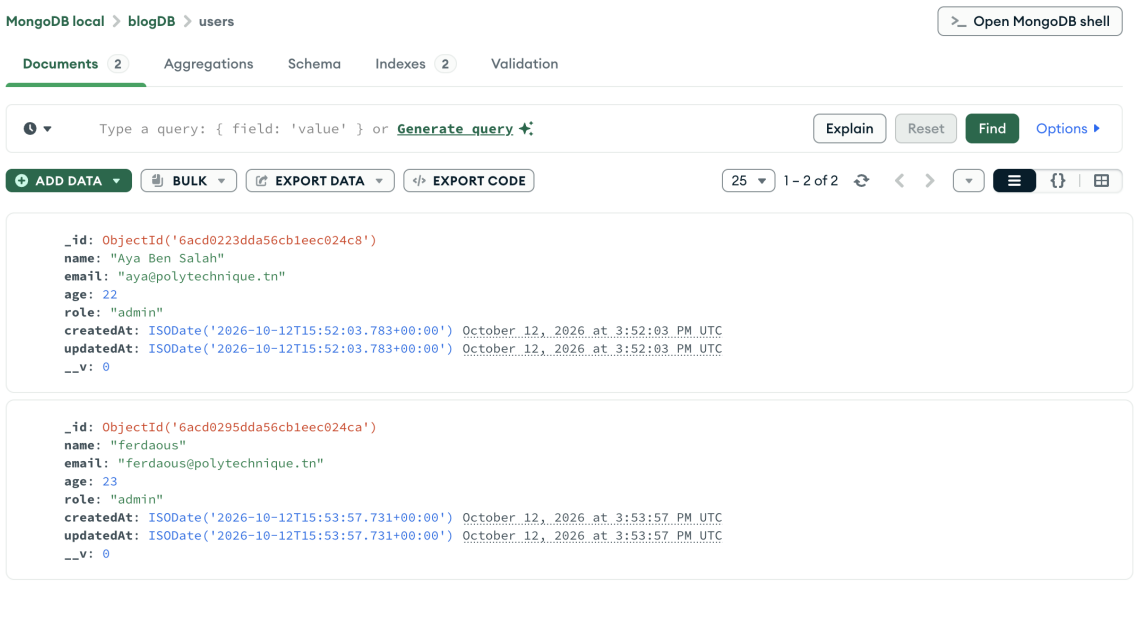
La base blogDB contient les collections users et articles. Les documents restent présents après le redémarrage du serveur.

Capture 6 - MongoDB Compass - aperçu de la base locale

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
articles	-	24.58 kB	185.00 B	1	185.00 B	1	24.58 kB
users	-	36.86 kB	300.00 B	2	150.00 B	2	73.73 kB

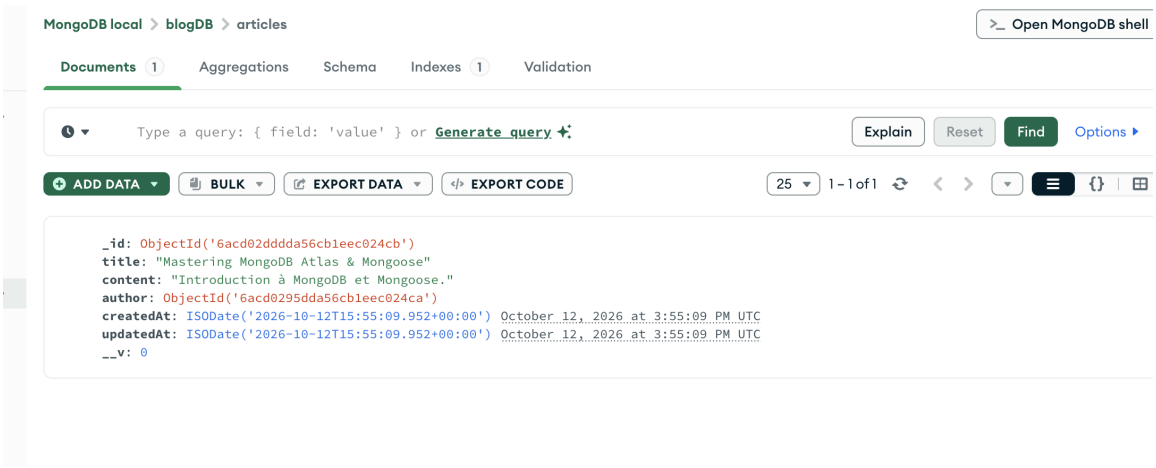
La base contient les collections articles et users avec respectivement un et deux documents.

Capture 7 - MongoDB Compass - collection users



La collection users contient les utilisateurs enregistrés.

Capture 8 - MongoDB Compass - collection articles



La collection articles contient l'article et l'ObjectId de son auteur.

Annexe A - Code complet des modèles

A.1 models/userModel.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [
        /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
        'Invalid email format (e.g., aya@polytechnique.tn)'
      ]
    },
    age: {
      type: Number,
      min: [4, 'Minimum age is 4 years old']
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'admin'],
        message: '{VALUE} is not an authorized role'
      },
      default: 'user'
    },
    bio: {
      type: String,
      maxlength: [200, 'Bio cannot exceed 200 characters']
    },
    phone: {
      type: String,
      match: [/^\d{8}$/, '8-digit phone number required']
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
```

A.2 models/articleModel.js

```
const mongoose = require('mongoose');

const articleSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Title is required'],
      trim: true
    },
    content: {
      type: String,
      required: [true, 'Content is required']
    },
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'Author (authorId) is required']
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model('Article', articleSchema);
```

Annexe B - Code complet des contrôleurs

B.1 controllers/userController.js

```
const User = require('../models/userModel');
const Article = require('../models/articleModel');

// POST /api/users
const createUser = async (req, res) => {
  try {
    const { name, email, age, role, bio, phone } = req.body;

    if (!name || !email) {
      return res.status(400).json({
        message: 'Name and email are required'
      });
    }

    const exists = await User.findOne({ email });

    if (exists) {
      return res.status(409).json({
        message: 'Email already in use'
      });
    }

    const user = await User.create({
      name,
      email,
      age,
      role,
      bio,
      phone
    });
    return res.status(201).json(user);
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ message: err.message });
    }

    if (err.code === 11000) {
      return res.status(409).json({ message: 'Email already in use' });
    }

    return res.status(500).json({ message: err.message });
  }
};

// GET /api/users
const getAllUsers = async (req, res) => {
  try {
    const users = await User.find();
    return res.status(200).json(users);
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// GET /api/users/:id
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);

    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    return res.status(200).json(user);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    return res.status(500).json({ message: err.message });
  }
};

// DELETE /api/users/:id
const deleteUser = async (req, res) => {
  try {
    const dependentArticle = await Article.findOne({ author: req.params.id });

    if (dependentArticle) {
      return res.status(400).json({
        message: 'Impossible to delete an author with published articles'
```

```
      });
    }

    const user = await User.findByIdAndDelete(req.params.id);

    if (!user) {
      return res.status(404).json({ message: 'User not found for deletion' });
    }

    return res.status(200).json({
      message: 'User deleted successfully',
      id: req.params.id
    });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    return res.status(500).json({ message: err.message });
  }
};

module.exports = {
  createUser,
  getAllUsers,
  getUserById,
  deleteUser
};
```

B.2 controllers/articleController.js

```
const Article = require('../models/articleModel');
const User = require('../models/userModel');

// GET /api/articles
const getAllArticles = async (req, res) => {
  try {
    const articles = await Article.find().populate('author', 'name email');
    return res.status(200).json(articles);
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
};

// POST /api/articles
const createArticle = async (req, res) => {
  try {
    const { title, content, authorId } = req.body;

    if (!title || !content || !authorId) {
      return res.status(400).json({
        message: 'title, content, and authorId are required'
      });
    }

    const author = await User.findById(authorId);

    if (!author) {
      return res.status(404).json({
        message: 'Author not found (invalid authorId)'
      });
    }

    const newArticle = await Article.create({
      title,
      content,
      author: authorId
    });

    const populatedArticle = await newArticle.populate(
      'author',
      'name email'
    );

    return res.status(201).json(populatedArticle);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    if (err.name === 'ValidationError') {
      return res.status(400).json({ message: err.message });
    }

    return res.status(500).json({ message: err.message });
  }
};

// GET /api/articles/:id
const getArticleById = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id).populate(
      'author',
      'name email'
    );

    if (!article) {
      return res.status(404).json({ message: 'Article not found' });
    }

    return res.status(200).json(article);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    return res.status(500).json({ message: err.message });
  }
};

// PUT /api/articles/:id
const updateArticle = async (req, res) => {
  try {
```

```
    const article = await Article.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!article) {
      return res.status(404).json({
        message: 'Article not found for update'
      });
    }

    const populatedArticle = await article.populate(
      'author',
      'name email'
    );

    return res.status(200).json(populatedArticle);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    if (err.name === 'ValidationError') {
      return res.status(400).json({ message: err.message });
    }

    return res.status(500).json({ message: err.message });
  }
};

// DELETE /api/articles/:id
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndDelete(req.params.id);

    if (!article) {
      return res.status(404).json({
        message: 'Article not found for deletion'
      });
    }

    return res.status(200).json({
      message: 'Article deleted successfully',
      id: req.params.id
    });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }

    return res.status(500).json({ message: err.message });
  }
};

module.exports = {
  getAllArticles,
  createArticle,
  getArticleById,
  updateArticle,
  deleteArticle
};
```